

关键算法设计说明文档

文档编号：ADS-ALG-001

版本：V1.0

编制日期：2025 年 6 月

密级：内部

文档说明：本文档为《控制回路性能评估、诊断与 PID 参数整定系统架构设计说明书》(ADS) 的正式补充文件，详细描述了系统核心算法的数学原理、计算步骤、参数定义、伪代码实现及复杂度分析。本文档面向系统开发团队、算法工程师及测试工程师。

编制依据： - GB/T 44693.2-2024 《危险化学品企业工艺平稳性 第 2 部分：控制回路性能评估与优化技术规范》 - GB/T 44693.1-2024 《危险化学品企业工艺平稳性 第 1 部分：管理导则》 - GB/T 31497 信息技术 安全技术 信息安全管理 监视、测量、分析和评价

目录

1. 算法设计背景与目标
 2. 算法原理与数学模型
 3. 装置级与回路级性能评估核心指标算法
 4. 控制回路诊断指标体系算法
 5. PID 参数整定算法
 6. 算法输入输出参数定义及数据类型规范
 7. 算法时间复杂度与空间复杂度分析
 8. 算法验证方法与预期结果
 9. 参考文献
-

1. 算法设计背景与目标

1.1 项目背景

在危险化学品生产企业中，基本过程控制系统 (BPCS) 中的控制回路是保障工艺平稳性的核心单元。据统计，工业过程控制中约 30%-60% 的 PID 回路存在性能不佳的问题 [1]。随着装置运行周期的延长和工况条件的变化，控制回路的性能会逐渐衰减。因此，对控制回路进行持续的性能评估、诊断与优化，是确保生产过程安全、高效运行的必要措施。

GB/T 44693.2-2024 标准的发布，为危险化学品企业控制回路性能评估与优化提供了统一的技术规范。本文档基于该标准，结合学术文献和工业最佳实践，设计了一套完整的核心算法体系。

1.2 设计目标

目标维度	具体指标	对应标准条款
实时性	单回路评估计算时间 < 1 秒（万级数据点）	6.2 数据采集及处理
准确性	评估指标偏差 < 5%，故障诊断准确率 > 85%	6.3 单回路评估
可靠性	系统可用性 > 99.9%，异常数据自动剔除	6.2.1 d)
合规性	完全符合 GB/T 44693.2-2024 附录 B-G 要求	全标准
可扩展性	支持千级回路并发评估，水平扩展架构	附录 H

1.3 适用范围

本算法设计适用于：- 危险化学品生产企业的装置基本过程控制系统 (BPCS) - 单回路 PID 控制器的性能评估与诊断 - 串级、分程、前馈等复杂控制回路的扩展评估 - PID 参数的离线与在线整定优化

1.4 术语与缩略语

本文档采用的术语与缩略语与 GB/T 44693.2-2024 第 3 章保持一致，补充如下：

缩略语	全称	说明
CPA	Controller Performance Assessment	控制器性能评估
MVC	Minimum Variance Control	最小方差控制
ARMA	Autoregressive Moving Average	自回归滑动平均
FOPDT	First Order Plus Dead Time	一阶加纯滞后模型
IMC	Internal Model Control	内模控制
SIMC	Simple IMC / Skogestad IMC	简化内模控制
NLI	Nonlinearity Index	非线性指数

2. 算法原理与数学模型

2.1 最小方差控制性能评估理论

2.1.1 Harris 指数原理 Harris 于 1989 年提出了基于最小方差控制 (MVC) 基准的控制回路性能评估方法 [1]，这是控制性能评估 (CPA) 领域的奠基性工作。

核心思想：对于线性过程，在最小方差控制器作用下，过程输出的方差达到理论最小值 σ^2_{MV} 。通过比较实际运行方差与理论最小方差的比值，可以量化当前控制器的性能：

$$\eta = \frac{\sigma^2_{MV}}{\sigma^2_y}$$

其中：- $\eta \in [0, 1]$ ：性能指数（越接近 1 表示性能越好）- σ^2_{MV} ：最小方差控制下的输出方差 - σ^2_y ：实际输出方差

2.1.2 基于闭环数据的 FCOR 算法 Huang 和 Shah[2] 提出的 FCOR(Filtering and COrrrelation) 算法，仅需闭环运行数据和过程时延即可评估性能。

数学推导：对于 SISO 闭环系统，输出 y_t 可表示为：

$$y_t = \sum_{i=0}^{d-1} f_i a_{t-i} + \sum_{i=d}^{\infty} f_i a_{t-i}$$

其中前 d 项（ d 为过程时延）为反馈控制不变项，其方差即为最小可达方差。

通过拟合 ARMA(p,q) 模型，利用长除法展开得到脉冲响应系数 f_i ，则：

$$\sigma^2_{MV} = \sum_{i=0}^{d-1} f_i^2 \cdot \sigma_a^2$$

$$\eta = \frac{\sum_{i=0}^{d-1} f_i^2}{\sum_{i=0}^{\infty} f_i^2}$$

2.2 控制回路振荡检测理论

2.2.1 自相关函数法 Hägglund[4] 提出了基于控制偏差自相关函数的振荡检测方法。对于控制偏差信号 $e(t)$ ，其自相关函数为：

$$r_e(k) = \frac{\sum_{t=1}^{N-k} (e_t - \bar{e})(e_{t+k} - \bar{e})}{\sum_{t=1}^N (e_t - \bar{e})^2}$$

衰减比计算：取自相关函数的第一个正峰值与零滞后的比值 $R_a = r_e(0)/r_e(k_1)$ ，衰减比 $DR = 1/R_a$ 。当 $DR < 0.5$ 时，表明存在显著振荡。

2.2.2 零交叉点正则性检测 Thornhill 等 [5] 提出通过分析零交叉点间隔的正则性来检测振荡。定义正则性指数：

$$r = \frac{\bar{T}_p}{3\sigma_T}$$

其中 $\bar{T}_p$ 为零交叉间隔均值， σ_T 为标准差。当 $r > 1$ 时，表明存在规律性振荡。

2.3 阀门粘滞检测理论

2.3.1 Kano 粘滞模型 Kano 等 [6] 提出了阀门粘滞的数据驱动模型, 定义两个关键参数: - **S** (死区 + 粘滞带): 阀门不响应控制器输出变化的范围 - **J** (滑跳量): 克服静摩擦力后阀门的突然移动量

当 $|OP(k) - OP(k-1)| \leq S$ 时, 阀门位置不变 (粘滞阶段); 当累计偏差超过 S 时, 阀门跳变 J (滑跳阶段)。

2.3.2 PV-OP 散点图分析法 通过绘制 PV 与 OP 的散点图, 正常线性系统的散点图呈近似线性分布; 存在粘滞时, 散点图呈现椭圆或平行四边形等非正常形状 [7]。

2.4 PID 整定理论基础

2.4.1 FOPDT 模型 工业过程大多可近似为一阶加纯滞后模型 (FOPDT):

$$G_p(s) = \frac{K}{Ts + 1} e^{-\theta s}$$

其中 K 为增益, T 为时间常数, θ 为纯滞后时间。

2.4.2 Lambda 整定法 Lambda 整定法通过指定期望的闭环时间常数 λ 来控制响应速度 [8]:

PI 控制器 (FOPDT 模型):

$$K_c = \frac{T}{K(\lambda + \theta)}$$
$$T_i = T$$

PID 控制器 (FOPDT 模型):

$$K_c = \frac{T + \theta/2}{K(\lambda + \theta/2)}$$
$$T_i = T + \frac{\theta}{2}$$
$$T_d = \frac{T\theta}{2T + \theta}$$

2.4.3 IMC-PID 整定法 基于内模控制原理, 对于 FOPDT 模型, IMC-PID 参数为 [9]:

$$K_c = \frac{T_I}{K_p(\varepsilon + \tau)}$$
$$T_I = T + \frac{\tau^2}{2(\varepsilon + \tau)}$$
$$T_D = \frac{\tau^2}{2(\varepsilon + \tau)} \left(1 - \frac{\tau}{3T_I} \right)$$

其中 ε 为 IMC 滤波器时间常数, τ 为等效时滞。

2.4.4 SIMC 整定法 Skogestad[8] 提出的 SIMC 方法是工业界广泛认可的 PID 整定方法:

PI 控制器:

$$K_c = \frac{T}{K(\tau_c + \theta)}$$

$$T_i = \min(T, 4(\tau_c + \theta))$$

PID 控制器 (串联形式):

$$K_c = \frac{T}{K(\tau_c + \theta)}$$

$$T_i = \min(T, 4(\tau_c + \theta))$$

$$T_d = T_2$$

其中 τ_c 为期望闭环时间常数 (推荐 $\tau_c = \theta$), T_2 为次要时间常数。

3. 性能评估核心指标算法

3.1 自控率算法 (AutoControlRate)

3.1.1 算法描述 自控率表示控制器处于自动状态时所占总时间的百分比, 是衡量控制回路投用情况的基础指标。

3.1.2 数学模型

$$Auto = \frac{AutoTime}{AllTime} \times 100\%$$

依据: GB/T 44693.2-2024 附录 B.1 公式 (B.1)

3.1.3 输入参数

参数名	类型	单位	说明
mode_sequence	float[]	-	控制模式时间序列 (1=AUTO, 0=MANUAL)
sample_interval	float	s	数据采样间隔
total_duration	float	s	统计时间范围总时长

3.1.4 输出参数

参数名	类型	单位	范围
auto_rate	float	%	[0, 100]

3.1.5 伪代码

Algorithm: AutoControlRate

Input: mode_sequence[n], sample_interval, total_duration

Output: auto_rate

```
1. auto_time ← 0
2. FOR i ← 0 TO n-1 DO
3.     IF mode_sequence[i] == 1 THEN // AUTO mode
4.         auto_time ← auto_time + sample_interval
5.     END IF
6. END FOR
7. auto_rate ← (auto_time / total_duration) × 100.0
8. RETURN auto_rate
```

3.1.6 复杂度分析

复杂度	值	说明
时间复杂度	O(n)	单次遍历
空间复杂度	O(1)	仅需常量空间

3.2 有效自控率算法 (EffectiveAutoRate)

3.2.1 算法描述 有效自控率表示控制器处于有效自动状态（输出不饱和且控制有效）时所占总时间的百分比。

3.2.2 数学模型

$$R = \frac{AutoRealTime}{AllTime} \times 100\%$$

依据：GB/T 44693.2-2024 附录 B.2 公式 (B.2)

3.2.3 输入参数

参数名	类型	单位	说明
mode_sequence	float[]	-	控制模式时间序列
op_sequence	float[]	%	输出值时间序列
op_high_limit	float	%	输出值上限
op_low_limit	float	%	输出值下限
sample_interval	float	s	采样间隔
saturation_margin	float	%	饱和判定裕度（默认 1%）

3.2.4 伪代码

Algorithm: EffectiveAutoRate

Input: mode_sequence[n], op_sequence[n], op_high_limit, op_low_limit,
sample_interval, saturation_margin

Output: effective_auto_rate

```
1. effective_time ← 0
2. sat_high ← op_high_limit - saturation_margin
3. sat_low ← op_low_limit + saturation_margin
4. FOR i ← 0 TO n-1 DO
5.     IF mode_sequence[i] == 1 THEN
6.         IF op_sequence[i] > sat_low AND op_sequence[i] < sat_high THEN
7.             effective_time ← effective_time + sample_interval
8.         END IF
9.     END IF
10. END FOR
11. total_time ← n × sample_interval
12. effective_auto_rate ← (effective_time / total_time) × 100.0
13. RETURN effective_auto_rate
```

3.2.5 复杂度分析

复杂度	值	说明
时间复杂度	O(n)	单次遍历
空间复杂度	O(1)	常量空间

3.3 准确率算法 (AccuracyRate)

3.3.1 算法描述 准确率用于衡量测量值达到设定值的程度，反映控制回路是否存在余差。

3.3.2 数学模型

$$A = \left[1 - \left(\frac{|\bar{E}|}{|\bar{E}|} \right) \times \left(1 - \frac{1}{e^{(|\bar{E}|/|E_{max}|)}} \right) \right] \times 100\%$$
$$|E_{max}| = \frac{\sum [\max(|E_i|) - |E_i|]}{n}$$
$$|\bar{E}| = \frac{\sum |E_i|}{n}$$
$$E_i = PV_i - SP_i$$

依据：GB/T 44693.2-2024 附录 B.3 公式 (B.3)

3.3.3 计算步骤 步骤 1: 计算每个时刻的控制偏差 $E_i = PV_i - SP_i$

步骤 2: 计算偏差绝对值的平均值 $|\bar{E}|$

步骤 3: 计算偏差绝对值偏离的平均值 $|E_{\max}|$

步骤 4: 代入公式计算准确率 A

3.3.4 输入参数

参数名	类型	单位	说明
pv_sequence	float[]	工程单位	过程测量值时间序列
sp_sequence	float[]	工程单位	设定值时间序列
n	int	-	数据点个数

3.3.5 输出参数

参数名	类型	单位	范围
accuracy_rate	float	%	[0, 100]
avg_deviation	float	工程单位	偏差绝对均值
max_deviation	float	工程单位	偏差绝对值偏离均值

3.3.6 伪代码

Algorithm: AccuracyRate

Input: pv_sequence[n], sp_sequence[n]

Output: accuracy_rate, avg_deviation, max_deviation

```
1. // 步骤1: 计算控制偏差
2. ALLOCATE deviation[n]
3. FOR i ← 0 TO n-1 DO
4.     deviation[i] ← pv_sequence[i] - sp_sequence[i]
5. END FOR
6.
7. // 步骤2: 计算偏差绝对值的平均值
8. sum_abs ← 0
9. FOR i ← 0 TO n-1 DO
10.    sum_abs ← sum_abs + ABS(deviation[i])
11. END FOR
12. avg_abs_dev ← sum_abs / n
13.
14. // 步骤3: 计算偏差绝对值偏离的平均值
15. max_abs_dev ← MAX(ABS(deviation[0..n-1]))
16. sum_deviation ← 0
17. FOR i ← 0 TO n-1 DO
18.    sum_deviation ← sum_deviation + (max_abs_dev - ABS(deviation[i]))
19. END FOR
```



```

20. E_max ← sum_deviation / n
21.
22. // 步骤4: 计算准确率
23. IF E_max > 0 THEN
24.     ratio ← avg_abs_dev / E_max
25.     accuracy_rate ← (1.0 - (1.0 - 1.0/EXP(ratio))) × 100.0
26. ELSE
27.     accuracy_rate ← 100.0 // 完美控制
28. END IF
29.
30. RETURN accuracy_rate, avg_abs_dev, E_max

```

3.3.7 复杂度分析

复杂度	值	说明
时间复杂度	O(n)	三次线性遍历
空间复杂度	O(n)	需存储偏差序列

3.4 快速率算法 (FastRate)

3.4.1 算法描述 快速率评估控制回路跟踪设定值变化的响应速度特性。核心是通过系统辨识获取实际稳态时间，与理想稳态时间比较。

3.4.2 数学模型

$$F = \frac{1}{e^{(T/T')}} \times 100\%$$

其中 T 为实际稳态时间（通过 ARMA 模型辨识获得），T' 为理想稳态时间。

依据：GB/T 44693.2-2024 附录 B.4 公式 (B.4)

3.4.3 实际稳态时间 T 的计算 (ARMA+Green 函数法)

步骤 1: ARMA 模型参数辨识 对控制回路闭环数据进行 ARMA(p,q) 建模：

$$x_t = \xi_t + \sum_{i=1}^p \phi_i x_{t-i} - \sum_{j=1}^q \theta_j \xi_{t-j}$$

采用最小二乘法估计参数 ϕ_i 和 θ_j 。

步骤 2: Green 函数计算 根据辨识参数计算单位脉冲响应函数 (Green 函数):

$$\begin{aligned}
 G_0 &= 1 \\
 G_1 &= \varphi_1 G_0 - \theta_1 \\
 G_2 &= \varphi_1 G_1 + \varphi_2 G_0 - \theta_2 \\
 &\dots \\
 G_k &= \varphi_1 G_{k-1} + \varphi_2 G_{k-2} + \dots + \varphi_p G_{k-p} \quad (\text{当 } k > \max(p, q))
 \end{aligned}$$

步骤 3: 稳态时间确定 找到 G_k 序列稳定在最终值 $\pm 5\%$ (或 $\pm 2\%$) 范围内的时刻 k_{stable} , 则:

$$T = k_{stable} \times \Delta t$$

3.4.4 输入参数

参数名	类型	单位	说明
pv_sequence	float[]	工程单位	过程测量值时间序列
sp_sequence	float[]	工程单位	设定值时间序列
sample_interval	float	s	采样间隔
p	int	-	AR 阶数 (默认 5)
q	int	-	MA 阶数 (默认 5)
ideal_settling_time	float	s	理想稳态时间

3.4.5 输出参数

参数名	类型	单位	范围
fast_rate	float	%	[0, 100]
actual_settling_time	float	s	>0
ar_coefficients	float[]	-	AR 系数
ma_coefficients	float[]	-	MA 系数

3.4.6 伪代码

Algorithm: FastRate

Input: pv_sequence[n], sp_sequence[n], sample_interval, p, q, ideal_settling_time

Output: fast_rate, actual_settling_time, ar_coeffs, ma_coeffs

1. // 步骤1: 计算控制偏差序列
2. ALLOCATE e[n]
3. FOR i ← 0 TO n-1 DO
4. e[i] ← pv_sequence[i] - sp_sequence[i]
5. END FOR
- 6.
7. // 步骤2: ARMA模型参数辨识 (最小二乘法)
8. (ar_coeffs, ma_coeffs) ← ARMA_LS_Estimate(e, p, q)

```

9.
10. // 步骤3: 计算Green函数 (脉冲响应)
11. num_impulse ← MAX(200, 10 × MAX(p, q))
12. ALLOCATE G[num_impulse]
13. G[0] ← 1.0
14. FOR k ← 1 TO num_impulse-1 DO
15.     G[k] ← 0
16.     FOR i ← 1 TO MIN(k, p) DO
17.         G[k] ← G[k] + ar_coeffs[i-1] × G[k-i]
18.     END FOR
19.     IF k ≤ q THEN
20.         G[k] ← G[k] - ma_coeffs[k-1]
21.     END IF
22. END FOR
23.
24. // 步骤4: 确定稳态时间 (5%收敛准则)
25. steady_value ← G[num_impulse-1]
26. settling_index ← num_impulse - 1
27. FOR k ← num_impulse-1 DOWNTO 0 DO
28.     IF ABS(G[k] - steady_value) > 0.05 × ABS(steady_value) THEN
29.         settling_index ← k + 1
30.         BREAK
31.     END IF
32. END FOR
33. actual_settling_time ← settling_index × sample_interval
34.
35. // 步骤5: 计算快速率
36. IF ideal_settling_time > 0 THEN
37.     fast_rate ← (1.0 / EXP(actual_settling_time / ideal_settling_time)) × 100.0
38. ELSE
39.     fast_rate ← 100.0
40. END IF
41.
42. RETURN fast_rate, actual_settling_time, ar_coeffs, ma_coeffs

```

3.4.7 ARMA 最小二乘辨识子算法

Algorithm: ARMA_LS_Estimate

Input: data_sequence[n], p, q

Output: ar_coeffs[p], ma_coeffs[q]

```

1. // 构建回归矩阵和观测向量
2. m ← MAX(p, q)
3. N ← n - m
4. ALLOCATE Phi[N × (p+q)], Y[N]
5.
6. FOR t ← m TO n-1 DO
7.     row ← t - m

```

```

8.     Y[row] ← data_sequence[t]
9.     FOR i ← 1 TO p DO
10.        Phi[row][i-1] ← data_sequence[t-i]
11.     END FOR
12.     // 注意：MA部分在初始估计中需要用残差迭代
13. END FOR
14.
15. // 步骤1：初始AR估计（忽略MA部分）
16. Phi_AR ← Phi[0..N-1][0..p-1]
17. ar_coeffs ← (Phi_AR^T × Phi_AR)^{-1} × Phi_AR^T × Y
18.
19. // 步骤2：计算残差
20. ALLOCATE residuals[N]
21. FOR t ← 0 TO N-1 DO
22.     predicted ← 0
23.     FOR i ← 1 TO p DO
24.         predicted ← predicted + ar_coeffs[i-1] × data_sequence[m+t-i]
25.     END FOR
26.     residuals[t] ← data_sequence[m+t] - predicted
27. END FOR
28.
29. // 步骤3：扩展回归矩阵包含MA部分
30. ALLOCATE Phi_ext[N × (p+q)]
31. FOR t ← 0 TO N-1 DO
32.     FOR i ← 1 TO p DO
33.         Phi_ext[t][i-1] ← data_sequence[m+t-i]
34.     END FOR
35.     FOR j ← 1 TO MIN(q, t) DO
36.         Phi_ext[t][p+j-1] ← -residuals[t-j]
37.     END FOR
38. END FOR
39.
40. // 步骤4：联合最小二乘估计
41. theta ← (Phi_ext^T × Phi_ext)^{-1} × Phi_ext^T × Y
42. ar_coeffs ← theta[0..p-1]
43. ma_coeffs ← theta[p..p+q-1]
44.
45. RETURN ar_coeffs, ma_coeffs

```

3.4.8 复杂度分析

复杂度	值	说明
时间复杂度	$O(n \cdot (p+q)^2)$	矩阵乘法为主
空间复杂度	$O(n \cdot (p+q))$	存储回归矩阵

3.5 稳定率算法 (StabilityRate)

3.5.1 算法描述 稳定率基于控制偏差的标准差进行评估，标准差越小，表明实际输出值在设定值附近的波动幅度越小。

3.5.2 数学模型

$$S = \left[\frac{1}{e^{(\sigma/(0.05 \times U))}} \right] \times (1 - Osc) \times 100\%$$

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (E_i - \bar{E})^2}{n - 1}}$$

$$E_i = PV_i - SP_i$$

依据：GB/T 44693.2-2024 附录 B.5 公式 (B.5)

3.5.3 输入参数

参数名	类型	单位	说明
pv_sequence	float[]	工程单位	过程测量值
sp_sequence	float[]	工程单位	设定值
instrument_range	float	工程单位	仪表量程 U
oscillation_rate	float	%	振荡率（来自 4.1 节算法）

3.5.4 输出参数

参数名	类型	单位	范围
stability_rate	float	%	[0, 100]
deviation_std	float	工程单位	\$\square\$0

3.5.5 伪代码

Algorithm: StabilityRate

Input: pv_sequence[n], sp_sequence[n], instrument_range, oscillation_rate

Output: stability_rate, deviation_std

1. // 步骤1: 计算控制偏差序列
2. sum_dev ← 0, sum_sq_dev ← 0
3. FOR i ← 0 TO n-1 DO
4. e_i ← pv_sequence[i] - sp_sequence[i]
5. sum_dev ← sum_dev + e_i
6. sum_sq_dev ← sum_sq_dev + e_i × e_i
7. END FOR
- 8.
9. // 步骤2: 计算标准差

```

10. mean_dev ← sum_dev / n
11. variance ← (sum_sq_dev - n × mean_dev × mean_dev) / (n - 1)
12. deviation_std ← SQRT(MAX(variance, 0))
13.
14. // 步骤3: 计算稳定率
15. normalized_std ← deviation_std / (0.05 × instrument_range)
16. IF normalized_std > 0 THEN
17.     stability_rate ← (1.0 / EXP(normalized_std)) × (1.0 -
oscillation_rate/100.0) × 100.0
18. ELSE
19.     stability_rate ← (1.0 - oscillation_rate/100.0) × 100.0
20. END IF
21.
22. // 边界保护
23. stability_rate ← CLAMP(stability_rate, 0.0, 100.0)
24.
25. RETURN stability_rate, deviation_std

```

3.5.6 复杂度分析

复杂度	值	说明
时间复杂度	O(n)	单次遍历
空间复杂度	O(1)	在线计算，无需存储完整序列

3.6 综合性能评分算法 (PerformanceScore)

3.6.1 算法描述 综合准确率 (A)、快速率 (F)、稳定率 (S) 和有效自控率 (R)，按权重加权计算控制回路的综合性能评分。

3.6.2 数学模型

$$P = \frac{\sum(A \times a + F \times f + S \times s)}{a + f + s} \times R$$

依据：GB/T 44693.2-2024 6.3.1 公式 (1)

3.6.3 权重系数配置 根据回路控制类型选择权重系数，依据 GB/T 44693.2-2024 附录 C：

控制类型	权重 a (准确率)	权重 f (快速率)	权重 s (稳定率)	典型场景
稳定型	0.2	0.3	0.5	温度控制、压力控制
慢速型	0.3	0.1	0.6	缓慢调节的工艺参数
快速型	0.2	0.5	0.3	副回路、速度控制

控制类型	权重 a（准确率）	权重 f（快速率）	权重 s（稳定率）	典型场景
逻辑型	0	0.4	0.6	安全动作、 联锁保护

3.6.4 性能定级标准 依据：GB/T 44693.2-2024 附录 D：

性能评分 P	性能定级	标识颜色
$90 \leq P < 100$	一级	绿色
$80 \leq P < 90$	二级	蓝色
$70 \leq P < 80$	三级	黄色
$60 \leq P < 70$	四级	橙色
$0 \leq P < 60$	五级	红色

3.6.5 输入参数

参数名	类型	单位	说明
accuracy_rate	float	%	准确率 A
fast_rate	float	%	快速率 F
stability_rate	float	%	稳定率 S
effective_auto_rate	float	%	有效自控率 R
control_type	enum	-	控制类型 (STABLE/SLOW/FAST/LOGIC)

3.6.6 输出参数

参数名	类型	单位	范围
performance_score	float	%	[0, 100]
performance_grade	int	-	1-5
grade_label	string	-	一级 ~ 五级

3.6.7 伪代码

Algorithm: PerformanceScore
Input: accuracy_rate, fast_rate, stability_rate, effective_auto_rate, control_type
Output: performance_score, performance_grade, grade_label

- ```
1. // 步骤1: 选择权重系数
2. SWITCH control_type DO
3. CASE STABLE:
4. a ← 0.2, f ← 0.3, s ← 0.5
5. CASE SLOW:
6. a ← 0.3, f ← 0.1, s ← 0.6
7. CASE FAST:
```

```

8. a ← 0.2, f ← 0.5, s ← 0.3
9. CASE LOGIC:
10. a ← 0.0, f ← 0.4, s ← 0.6
11. END SWITCH
12.
13. // 步骤2: 计算综合评分
14. numerator ← accuracy_rate × a + fast_rate × f + stability_rate × s
15. denominator ← a + f + s
16. IF denominator > 0 THEN
17. performance_score ← (numerator / denominator) × (effective_auto_rate / 100.0)
18. ELSE
19. performance_score ← 0
20. END IF
21.
22. // 步骤3: 性能定级
23. IF performance_score ≥ 90 THEN
24. performance_grade ← 1, grade_label ← "一级"
25. ELSE IF performance_score ≥ 80 THEN
26. performance_grade ← 2, grade_label ← "二级"
27. ELSE IF performance_score ≥ 70 THEN
28. performance_grade ← 3, grade_label ← "三级"
29. ELSE IF performance_score ≥ 60 THEN
30. performance_grade ← 4, grade_label ← "四级"
31. ELSE
32. performance_grade ← 5, grade_label ← "五级"
33. END IF
34.
35. RETURN performance_score, performance_grade, grade_label

```

### 3.6.8 复杂度分析

| 复杂度   | 值    | 说明   |
|-------|------|------|
| 时间复杂度 | O(1) | 常数时间 |
| 空间复杂度 | O(1) | 常数空间 |

## 3.7 装置级 KPI 加权聚合算法 (UnitLevelAggregation)

**3.7.1 算法描述** 将装置内所有回路的性能指标按回路权重加权平均，得到装置级关键绩效指标。

### 3.7.2 数学模型

$$P_{unit} = \frac{\sum_{i=1}^m w_i \times P_i}{\sum_{i=1}^m w_i}$$

回路权重（依据 GB/T 44693.2-2024 附录 E.2）：



| 回路级别 | 权重 w | 描述                    |
|------|------|-----------------------|
| 一级回路 | 3    | 对装置整体性能/安全性/经济性有决定性影响 |
| 二级回路 | 2    | 对装置运行稳定性或主要设备安全有较大影响  |
| 三级回路 | 1    | 相对次要，维持辅助设备正常运行       |

### 3.7.3 装置级 KPI 管理目标 依据：GB/T 44693.2-2024 附录 E.1:

| 指标    | 参考目标  |
|-------|-------|
| 实时自控率 | ≥ 90% |
| 平均自控率 | ≥ 95% |
| 平稳率   | ≥ 95% |
| 性能评分  | ≥ 80% |

### 3.7.4 伪代码

Algorithm: UnitLevelAggregation

Input: loop\_scores[m], loop\_weights[m]

Output: unit\_score, unit\_auto\_rate, unit\_stability\_rate

```

1. weighted_score_sum ← 0, weight_sum ← 0
2. weighted_auto_sum ← 0, weighted_stability_sum ← 0
3.
4. FOR i ← 0 TO m-1 DO
5. w ← loop_weights[i]
6. weighted_score_sum ← weighted_score_sum + w × loop_scores[i].performance_score
7. weighted_auto_sum ← weighted_auto_sum + w × loop_scores[i].auto_rate
8. weighted_stability_sum ← weighted_stability_sum + w × loop_scores[i].stability_rate
9. weight_sum ← weight_sum + w
10. END FOR
11.
12. IF weight_sum > 0 THEN
13. unit_score ← weighted_score_sum / weight_sum
14. unit_auto_rate ← weighted_auto_sum / weight_sum
15. unit_stability_rate ← weighted_stability_sum / weight_sum
16. ELSE
17. unit_score ← 0; unit_auto_rate ← 0; unit_stability_rate ← 0
18. END IF
19.
20. RETURN unit_score, unit_auto_rate, unit_stability_rate

```

4. 控制回路诊断指标体系算法

4.1 振荡率检测算法 (OscillationDetection)

4.1.1 算法描述 振荡率用于衡量控制回路测量值振荡程度的指标。通过分析数据序列的持续正/负值面积和持续时间的相似性来评估振荡情况。

依据：GB/T 44693.2-2024 附录 F.1

4.1.2 数学模型

$$O_{osc} = \frac{As_{osc} + At_{osc} + Bs_{osc} + Bt_{osc}}{4}$$

其中：- As\_osc：持续为正值的面积值相似率 - At\_osc：持续为正值的持续时间相似率 - Bs\_osc：持续为负值的面积值相似率 - Bt\_osc：持续为负值的持续时间相似率

4.1.3 最小距离法相似率计算 对于数据序列  $K_i, m>10$ :

步骤 a：最小距离法计算平均值

$$avg = K_{imin} \quad \text{满足} \quad \sum_{i=1}^m (K_i - K_{imin})^2 \leq \sum_{i=1}^m (K_i - K_j)^2 \quad (j = 1, 2, \dots, m)$$

步骤 b：清除异常数据——如果  $0.5 \geq (K_i/K_{imin})$  或  $1.5 \leq (K_i/K_{imin})$ ，则清除该数据

步骤 c：重新计算平均值并计算相似率

$$S_i = \frac{\sum_{i=1}^z \frac{\min(Kn_i, avgn)}{\max(Kn_i, avgn)}}{m} \times 100\%$$

4.1.4 输入参数

| 参数名                | 类型      | 单位   | 说明                                         |
|--------------------|---------|------|--------------------------------------------|
| deviation_sequence | float[] | 工程单位 | 控制偏差序列 $E_i = PV_i - SP_i$<br>数据点个数（需 >10） |
| n                  | int     | -    |                                            |

4.1.5 输出参数

| 参数名              | 类型    | 单位 | 范围       |
|------------------|-------|----|----------|
| oscillation_rate | float | %  | [0, 100] |
| as_osc           | float | %  | 正面积相似率   |
| at_osc           | float | %  | 正持续时间相似率 |

| 参数名    | 类型    | 单位 | 范围       |
|--------|-------|----|----------|
| bs_osc | float | %  | 负面积相似率   |
| bt_osc | float | %  | 负持续时间相似率 |

#### 4.1.6 伪代码

Algorithm: OscillationDetection

Input: deviation\_sequence[n]

Output: oscillation\_rate, as\_osc, at\_osc, bs\_osc, bt\_osc

```

1. // 步骤1: 提取持续正值段和负值段
2. ALLOCATE pos_areas[], pos_durations[], neg_areas[], neg_durations[]
3. current_area ← 0, current_duration ← 0
4. current_sign ← SIGN(deviation_sequence[0])
5.
6. FOR i ← 0 TO n-1 DO
7. IF SIGN(deviation_sequence[i]) == current_sign THEN
8. current_area ← current_area + ABS(deviation_sequence[i])
9. current_duration ← current_duration + 1
10. ELSE
11. // 保存当前段
12. IF current_sign > 0 THEN
13. APPEND pos_areas WITH current_area
14. APPEND pos_durations WITH current_duration
15. ELSE IF current_sign < 0 THEN
16. APPEND neg_areas WITH current_area
17. APPEND neg_durations WITH current_duration
18. END IF
19. current_sign ← SIGN(deviation_sequence[i])
20. current_area ← ABS(deviation_sequence[i])
21. current_duration ← 1
22. END IF
23. END FOR
24.
25. // 步骤2: 对各序列计算相似率
26. as_osc ← SimilarityRate(pos_areas)
27. at_osc ← SimilarityRate(pos_durations)
28. bs_osc ← SimilarityRate(neg_areas)
29. bt_osc ← SimilarityRate(neg_durations)
30.
31. // 步骤3: 综合振荡率
32. oscillation_rate ← (as_osc + at_osc + bs_osc + bt_osc) / 4.0
33. oscillation_rate ← CLAMP(oscillation_rate, 0.0, 100.0)
34.
35. RETURN oscillation_rate, as_osc, at_osc, bs_osc, bt_osc

```

#### 4.1.7 相似率子算法 (SimilarityRate)

Algorithm: SimilarityRate

Input: data\_sequence[m]

Output: similarity\_rate

```
1. IF m < 10 THEN
2. RETURN 0.0 // 数据不足
3. END IF
4.
5. // 步骤a: 最小距离法计算平均值
6. min_sq_sum ← +∞
7. avg_idx ← 0
8. FOR j ← 0 TO m-1 DO
9. sq_sum ← 0
10. FOR i ← 0 TO m-1 DO
11. sq_sum ← sq_sum + (data_sequence[i] - data_sequence[j])2
12. END FOR
13. IF sq_sum < min_sq_sum THEN
14. min_sq_sum ← sq_sum
15. avg_idx ← j
16. END IF
17. END FOR
18. avg ← data_sequence[avg_idx]
19.
20. // 步骤b: 清除异常数据
21. IF avg == 0 THEN
22. RETURN 0.0
23. END IF
24.
25. ALLOCATE cleaned_data[]
26. FOR i ← 0 TO m-1 DO
27. ratio ← data_sequence[i] / avg
28. IF ratio >= 0.5 AND ratio <= 1.5 THEN
29. APPEND cleaned_data WITH data_sequence[i]
30. END IF
31. END FOR
32.
33. z ← LENGTH(cleaned_data)
34. IF z == 0 THEN
35. RETURN 0.0
36. END IF
37.
38. // 步骤c: 重新计算平均值和相似率
39. avg_n ← SimilarityRate_Avg(cleaned_data) // 递归调用最小距离法
40.
41. sim_sum ← 0
42. FOR i ← 0 TO z-1 DO
```

```

43. sim_sum ← sim_sum + MIN(cleaned_data[i], avg_n) / MAX(cleaned_data[i], avg_n)
44. END FOR
45.
46. similarity_rate ← (sim_sum / m) × 100.0
47. RETURN similarity_rate

```

#### 4.1.8 复杂度分析

| 复杂度   | 值            | 说明                               |
|-------|--------------|----------------------------------|
| 时间复杂度 | $O(n + m^2)$ | 遍历数据 + 最小距离法 (m 为段数, $m \ll n$ ) |
| 空间复杂度 | $O(m)$       | 存储正负段信息                          |

### 4.2 黏滞系数检测算法 (StictionDetection)

**4.2.1 算法描述** 黏滞系数用于量化控制回路输出值的黏滞特性。基于控制偏差及阀门输出偏差的振荡率，结合异常波形振荡率计算。

依据：GB/T 44693.2-2024 附录 F.2

#### 4.2.2 数学模型

$$St = \max(E_f, O_f) \times \sqrt{E_r \times O_r}$$

其中：-  $E_f$ ：控制偏差异常波形振荡率 -  $O_f$ ：阀门输出偏差异常波形振荡率 -  $E_r$ ：控制偏差振荡率 -  $O_r$ ：阀门输出偏差振荡率

#### 4.2.3 异常波形振荡率计算 方形波形异常振荡率：

$$F_s = \frac{A_o + B_o}{2}$$

当正值序列的平均值相似率  $A_o > 40$  且负值序列的平均值相似率  $B_o > 40$  时，判定为方形波形。

三角形波形异常振荡率：

$$F_t = 1 - \frac{\min(Amt_a, Bmt_a)}{\max(Amt_a, Bmt_a)}$$

综合异常振荡率：

$$F_f = \max(F_s, F_t)$$

#### 4.2.4 输入参数

| 参数名         | 类型      | 单位   | 说明    |
|-------------|---------|------|-------|
| pv_sequence | float[] | 工程单位 | 过程测量值 |
| sp_sequence | float[] | 工程单位 | 设定值   |

| 参数名         | 类型      | 单位 | 说明  |
|-------------|---------|----|-----|
| op_sequence | float[] | %  | 输出值 |

#### 4.2.5 输出参数

| 参数名                  | 类型    | 单位 | 范围          |
|----------------------|-------|----|-------------|
| stiction_coefficient | float | %  | [0, 100]    |
| error_abnormal_osc   | float | %  | 偏差异常波形振荡率   |
| op_abnormal_osc      | float | %  | 输出偏差异常波形振荡率 |

#### 4.2.6 伪代码

Algorithm: StictionDetection

Input: pv\_sequence[n], sp\_sequence[n], op\_sequence[n]

Output: stiction\_coefficient, error\_abnormal\_osc, op\_abnormal\_osc

```

1. // 步骤1: 计算控制偏差和输出偏差序列
2. ALLOCATE e[n], de_op[n]
3. FOR i ← 0 TO n-1 DO
4. e[i] ← pv_sequence[i] - sp_sequence[i]
5. END FOR
6. FOR i ← 1 TO n-1 DO
7. de_op[i] ← op_sequence[i] - op_sequence[i-1]
8. END FOR
9.
10. // 步骤2: 计算常规振荡率
11. e_osc ← OscillationDetection(e)
12. op_osc ← OscillationDetection(de_op[1..n-1])
13.
14. // 步骤3: 计算异常波形振荡率
15. e_abnormal ← AbnormalOscillationRate(e)
16. op_abnormal ← AbnormalOscillationRate(de_op[1..n-1])
17.
18. // 步骤4: 计算黏滞系数
19. max_abnormal ← MAX(e_abnormal, op_abnormal)
20. stiction_coefficient ← max_abnormal × SQRT(e_osc × op_osc)
21. stiction_coefficient ← CLAMP(stiction_coefficient, 0.0, 100.0)
22.
23. RETURN stiction_coefficient, e_abnormal, op_abnormal

```

#### 4.2.7 异常波形振荡率子算法

Algorithm: AbnormalOscillationRate

Input: data\_sequence[n]

Output: abnormal\_rate

```

1. // 提取正负值序列
2. pos_values[], neg_values[] ← split_by_sign(data_sequence)
3. pos_peaks[], neg_peaks[] ← extract_peak_durations(data_sequence)
4.
5. // 计算方形波形异常振荡率
6. Ao ← SimilarityRate(pos_values)
7. Bo ← SimilarityRate(neg_values)
8. IF Ao > 40 AND Bo > 40 THEN
9. Fs ← (Ao + Bo) / 2.0
10. ELSE
11. Fs ← 0
12. END IF
13.
14. // 计算三角形波形异常振荡率
15. Amta ← MinimumDistanceAvg(pos_peaks)
16. Bmta ← MinimumDistanceAvg(neg_peaks)
17. IF Amta > 40 AND Bmta > 40 THEN
18. Ft ← 1.0 - MIN(Amta, Bmta) / MAX(Amta, Bmta)
19. ELSE
20. Ft ← 0
21. END IF
22.
23. // 取最大值
24. abnormal_rate ← MAX(Fs, Ft)
25. RETURN abnormal_rate

```

#### 4.3 饱和率诊断算法 (SaturationRate)

**4.3.1 算法描述** 根据输出值处于限位的时间占统计总时间的百分比评估执行器（调节阀）选型是否合理。

依据：GB/T 44693.2-2024 附录 F.3 公式 (F.8)

##### 4.3.2 数学模型

$$Sa = \frac{AutoSaturateTime}{AllTime} \times 100\%$$

##### 4.3.3 伪代码

Algorithm: SaturationRate

Input: op\_sequence[n], mode\_sequence[n], op\_high\_limit, op\_low\_limit,  
sample\_interval, saturation\_margin

Output: saturation\_rate

```

1. saturate_time ← 0
2. sat_high ← op_high_limit - saturation_margin
3. sat_low ← op_low_limit + saturation_margin

```

```

4.
5. FOR i ← 0 TO n-1 DO
6. IF mode_sequence[i] == 1 THEN // AUTO模式
7. IF op_sequence[i] >= sat_high OR op_sequence[i] <= sat_low THEN
8. saturate_time ← saturate_time + sample_interval
9. END IF
10. END IF
11. END FOR
12.
13. total_time ← n × sample_interval
14. saturation_rate ← (saturate_time / total_time) × 100.0
15. RETURN saturation_rate

```

#### 4.4 输出值行程指数算法 (TravelIndex)

**4.4.1 算法描述** 表示平均每小时输出值的运动里程，用于评估阀门磨损情况。

依据：GB/T 44693.2-2024 附录 F.5 公式 (F.11)

##### 4.4.2 数学模型

$$Trip = \frac{\sum_{i=2}^n |Op_i - Op_{i-1}|}{t} \times 100\%$$

##### 4.4.3 伪代码

Algorithm: TravelIndex

Input: op\_sequence[n], sample\_interval

Output: travel\_index

```

1. total_travel ← 0
2. FOR i ← 1 TO n-1 DO
3. total_travel ← total_travel + ABS(op_sequence[i] - op_sequence[i-1])
4. END FOR
5.
6. total_hours ← n × sample_interval / 3600.0
7. IF total_hours > 0 THEN
8. travel_index ← (total_travel / total_hours) × 100.0
9. ELSE
10. travel_index ← 0
11. END IF
12.
13. RETURN travel_index

```

#### 4.5 好值率诊断算法 (QualityRate)

**4.5.1 算法描述** 用于表征控制回路执行器与仪表存在的异常程度，通过数据质量评估得出。

依据：GB/T 44693.2-2024 附录 F.6 公式 (F.12)



#### 4.5.2 伪代码

Algorithm: QualityRate

Input: quality\_flags[n], sample\_interval

Output: quality\_rate

```
1. good_time ← 0
2. FOR i ← 0 TO n-1 DO
3. IF quality_flags[i] IS VALID THEN // 质量码正常
4. good_time ← good_time + sample_interval
5. END IF
6. END FOR
7.
8. total_time ← n × sample_interval
9. quality_rate ← (good_time / total_time) × 100.0
10. RETURN quality_rate
```

#### 4.6 相关性指数算法 (CorrelationAnalysis)

**4.6.1 算法描述** 基于皮尔逊相关系数，衡量不同控制回路各参数变量之间的关联程度。

依据：GB/T 44693.2-2024 附录 F.7 公式 (F.13)

##### 4.6.2 数学模型

$$Sim = \frac{\sum_{i=1}^n (Ns_i - \bar{Ns})(Os_i - \bar{Os})}{\sqrt{\sum_{i=1}^n (Ns_i - \bar{Ns})^2 \sum_{i=1}^n (Os_i - \bar{Os})^2}}$$

##### 4.6.3 伪代码

Algorithm: CorrelationAnalysis

Input: sequence\_a[n], sequence\_b[n]

Output: correlation\_coefficient

```
1. // 计算均值
2. mean_a ← MEAN(sequence_a)
3. mean_b ← MEAN(sequence_b)
4.
5. // 计算协方差和方差
6. cov ← 0, var_a ← 0, var_b ← 0
7. FOR i ← 0 TO n-1 DO
8. diff_a ← sequence_a[i] - mean_a
9. diff_b ← sequence_b[i] - mean_b
10. cov ← cov + diff_a × diff_b
11. var_a ← var_a + diff_a × diff_a
12. var_b ← var_b + diff_b × diff_b
13. END FOR
14.
```

```

15. // 计算相关系数
16. IF var_a > 0 AND var_b > 0 THEN
17. correlation_coefficient ← cov / SQRT(var_a × var_b)
18. ELSE
19. correlation_coefficient ← 0
20. END IF
21.
22. RETURN correlation_coefficient

```

## 4.7 多指标综合诊断方法

**4.7.1 诊断逻辑** 依据 GB/T 44693.2-2024 附录 F.8，综合运用多个诊断指标进行故障定位：

| 故障类型         | 主要判断指标         | 辅助判断指标     | 判定条件            |
|--------------|----------------|------------|-----------------|
| PID 参数不当（振荡） | 振荡率 > 30%      | 黏滞系数 < 20% | 存在规律振荡且阀门无粘滞    |
| 阀门粘滞         | 黏滞系数 > 30%     | 振荡率 > 20%  | PV-OP 散点图呈非正常形状 |
| 阀门选型不当       | 饱和率 > 10%      | 行程指数异常     | 频繁饱和            |
| 仪表故障         | 好值率 < 90%      | 偏差突跳       | 数据质量异常          |
| 回路耦合         | 相关性指数 > 0.7    | —          | 与其他回路强相关        |
| 响应迟缓         | 稳态时间 > 3 × 理想值 | 快速率 < 50%  | 跟踪设定值变化缓慢       |

## 5. PID 参数整定算法

### 5.1 FOPDT 模型辨识算法

**5.1.1 算法描述** 通过阶跃响应数据辨识一阶加纯滞后 (FOPDT) 模型参数：增益  $K$ 、时间常数  $T$ 、纯滞后时间  $\theta$ 。

**5.1.2 辨识方法** 方法一：两点法（面积法）

$$\begin{aligned}
 K &= \frac{\Delta y}{\Delta u} \\
 T &= \frac{A_0}{K \Delta u} \\
 \theta &= \frac{\int_0^\infty t \cdot y'(t) dt}{\int_0^\infty y'(t) dt} - T
 \end{aligned}$$

其中  $A_0$  为阶跃响应曲线与稳态值之间的面积。

方法二：最小二乘法

将 FOPDT 模型的阶跃响应离散化，构造回归方程进行最小二乘拟合。

### 5.1.3 输入参数

| 参数名             | 类型      | 单位   | 说明              |
|-----------------|---------|------|-----------------|
| time_sequence   | float[] | s    | 时间序列            |
| pv_response     | float[] | 工程单位 | 阶跃响应数据          |
| step_magnitude  | float   | 工程单位 | 阶跃幅值 $\Delta u$ |
| initial_value   | float   | 工程单位 | 初始稳态值           |
| sample_interval | float   | s    | 采样间隔            |

### 5.1.4 输出参数

| 参数名         | 类型    | 单位            | 说明         |
|-------------|-------|---------------|------------|
| K           | float | (工程单位)/(工程单位) | 过程增益       |
| T           | float | s             | 时间常数       |
| theta       | float | s             | 纯滞后时间      |
| fit_quality | float | %             | 拟合优度 $R^2$ |

### 5.1.5 伪代码

Algorithm: FOPDT\_Identification

Input: time\_sequence[n], pv\_response[n], step\_magnitude, initial\_value, sample\_interval

Output: K, T, theta, fit\_quality

```

1. // 步骤1: 计算稳态增益
2. final_value ← pv_response[n-1]
3. delta_y ← final_value - initial_value
4. K ← delta_y / step_magnitude
5.
6. // 步骤2: 归一化响应
7. ALLOCATE y_norm[n]
8. FOR i ← 0 TO n-1 DO
9. y_norm[i] ← (pv_response[i] - initial_value) / delta_y
10. END FOR
11.
12. // 步骤3: 计算面积 (求时间常数和滞后)
13. // 找到响应达到28.3%和63.2%的时间点
14. t28 ← FindTimeAtValue(y_norm, 0.283)
15. t63 ← FindTimeAtValue(y_norm, 0.632)
16.
17. IF t28 > 0 AND t63 > 0 THEN
18. T ← 1.5 × (t63 - t28)
19. theta ← t63 - T
20. ELSE

```

```

21. // 使用面积法
22. area ← 0
23. FOR i ← 1 TO n-1 DO
24. area ← area + (1.0 - y_norm[i]) × sample_interval
25. END FOR
26. T ← area
27. theta ← 0
28. // 找到响应明显上升的时刻
29. FOR i ← 1 TO n-1 DO
30. IF y_norm[i] > 0.05 THEN
31. theta ← time_sequence[i] - T
32. BREAK
33. END IF
34. END FOR
35. END IF
36.
37. // 步骤4: 参数边界检查
38. theta ← MAX(theta, 0)
39. T ← MAX(T, sample_interval)
40.
41. // 步骤5: 计算拟合优度
42. fit_quality ← ComputeR2(y_norm, K, T, theta, time_sequence)
43.
44. RETURN K, T, theta, fit_quality

```

## 5.2 Lambda 整定算法 (LambdaTuning)

**5.2.1 算法描述** Lambda 整定法通过指定期望的闭环时间常数  $\lambda$ , 平衡响应速度和系统稳定性。

**5.2.2 数学模型 PI 控制器 (自衡对象 FOPDT):**

$$K_c = \frac{T}{K(\lambda + \theta)}$$

$$T_i = T$$

**PID 控制器 (自衡对象 FOPDT):**

$$K_c = \frac{T + \theta/2}{K(\lambda + \theta/2)}$$

$$T_i = T + \frac{\theta}{2}$$

$$T_d = \frac{T\theta}{2T + \theta}$$

**Lambda 值选择建议:** - 流量回路:  $\lambda = \theta$  (快速响应) - 压力回路:  $\lambda = 2\theta$  (中等响应) - 温度回路:  $\lambda = 3\theta$  (慢速响应) - 液位回路:  $\lambda = 5\theta$  (平滑控制)

### 5.2.3 输入参数

| 参数名             | 类型    | 单位 | 说明                           |
|-----------------|-------|----|------------------------------|
| K               | float | -  | 过程增益                         |
| T               | float | s  | 时间常数                         |
| theta           | float | s  | 纯滞后时间                        |
| lambda_factor   | float | -  | $\lambda/\theta$ 倍数 (默认 1-5) |
| controller_type | enum  | -  | PI / PID                     |

### 5.2.4 输出参数

| 参数名 | 类型    | 单位 | 说明           |
|-----|-------|----|--------------|
| Kc  | float | -  | 比例增益         |
| Ti  | float | s  | 积分时间         |
| Td  | float | s  | 微分时间 (仅 PID) |

### 5.2.5 伪代码

Algorithm: LambdaTuning

Input: K, T, theta, lambda\_factor, controller\_type

Output: Kc, Ti, Td

```
1. lambda ← lambda_factor × theta
2.
3. IF controller_type == PI THEN
4. Kc ← T / (K × (lambda + theta))
5. Ti ← T
6. Td ← 0
7. ELSE IF controller_type == PID THEN
8. Kc ← (T + theta/2.0) / (K × (lambda + theta/2.0))
9. Ti ← T + theta / 2.0
10. Td ← (T × theta) / (2.0 × T + theta)
11. END IF
12.
13. // 参数合理性检查
14. Kc ← MAX(Kc, 0.001)
15. Ti ← MAX(Ti, 0.1)
16. Td ← MAX(Td, 0)
17.
18. RETURN Kc, Ti, Td
```

## 5.3 IMC-PID 整定算法 (IMCPIDTuning)

**5.3.1 数学模型** 基于内模控制原理, 对于 FOPDT 模型:

$$K_c = \frac{T_I}{K_p(\varepsilon + \tau)}$$

$$T_I = T + \frac{\tau^2}{2(\varepsilon + \tau)}$$

$$T_D = \frac{\tau^2}{2(\varepsilon + \tau)} \left( 1 - \frac{\tau}{3T_I} \right)$$

其中  $\varepsilon$  为 IMC 滤波器时间常数 ( $\varepsilon \in [T/20, +\infty)$ ), 建议取  $\tau \leq \varepsilon \leq 3\tau$ 。

### 5.3.2 伪代码

Algorithm: IMCPIDTuning

Input: K, T, tau, epsilon\_factor

Output: Kc, Ti, Td

```

1. epsilon ← epsilon_factor × tau // 默认epsilon_factor = 1.0
2. epsilon ← MAX(epsilon, T / 20.0) // 下界约束
3.
4. // 计算IMC-PID参数
5. Ti ← T + tau2 / (2.0 × (epsilon + tau))
6. Kc ← Ti / (K × (epsilon + tau))
7. Td ← (tau2 / (2.0 × (epsilon + tau))) × (1.0 - tau / (3.0 × Ti))
8.
9. // 参数合理性检查
10. Kc ← MAX(Kc, 0.001)
11. Ti ← MAX(Ti, 0.1)
12. Td ← MAX(Td, 0)
13.
14. RETURN Kc, Ti, Td

```

## 5.4 SIMC 整定算法 (SIMCTuning)

5.4.1 数学模型 Skogestad[8] 提出的 SIMC 方法:

PI 控制器:

$$K_c = \frac{T}{K(\tau_c + \theta)}$$

$$T_i = \min(T, 4(\tau_c + \theta))$$

PID 控制器 (串联形式):

$$K_c = \frac{T}{K(\tau_c + \theta)}$$

$$T_i = \min(T, 4(\tau_c + \theta))$$

$$T_d = T_2$$

其中  $\tau_c$  为期望闭环时间常数（推荐  $\tau_c = \theta$ ）。

#### 5.4.2 伪代码

Algorithm: SIMCTuning

Input:  $K$ ,  $T$ ,  $\theta$ ,  $T_2$ ,  $\tau_{c\_factor}$ ,  $controller\_type$

Output:  $K_c$ ,  $T_i$ ,  $T_d$

```

1. $\tau_c \leftarrow \tau_{c_factor} \times \theta$ // 默认 $\tau_{c_factor} = 1.0$
2.
3. $K_c \leftarrow T / (K \times (\tau_c + \theta))$
4. $T_i \leftarrow \text{MIN}(T, 4.0 \times (\tau_c + \theta))$
5.
6. IF $controller_type == \text{PID}$ THEN
7. $T_d \leftarrow T_2$ // 次要时间常数
8. ELSE
9. $T_d \leftarrow 0$
10. END IF
11.
12. // 参数合理性检查
13. $K_c \leftarrow \text{MAX}(K_c, 0.001)$
14. $T_i \leftarrow \text{MAX}(T_i, 0.1)$
15. $T_d \leftarrow \text{MAX}(T_d, 0)$
16.
17. RETURN K_c , T_i , T_d

```

### 5.5 Ziegler-Nichols 工程整定算法 (ZNTuning)

5.5.1 临界振荡法 通过闭环试验找到临界增益  $K_u$  和临界周期  $P_u$ :

| 控制器类型 | $K_c$      | $T_i$     | $T_d$   |
|-------|------------|-----------|---------|
| P     | $0.5 K_u$  | —         | —       |
| PI    | $0.45 K_u$ | $P_u/1.2$ | —       |
| PID   | $0.6 K_u$  | $P_u/2$   | $P_u/8$ |

#### 5.5.2 阶跃响应法（基于 FOPDT 模型）

| 控制器类型 | $K_c$                   | $T_i$        | $T_d$       |
|-------|-------------------------|--------------|-------------|
| P     | $T/(K \cdot \theta)$    | —            | —           |
| PI    | $0.9T/(K \cdot \theta)$ | $3.33\theta$ | —           |
| PID   | $1.2T/(K \cdot \theta)$ | $2\theta$    | $0.5\theta$ |

#### 5.5.3 伪代码

Algorithm: ZNTuning

Input: K, T, theta, controller\_type, method // method: STEP or ULTIMATE  
Output: Kc, Ti, Td

```

1. IF method == STEP THEN
2. SWITCH controller_type DO
3. CASE P:
4. $K_c \leftarrow T / (K \times \theta)$; $T_i \leftarrow \infty$; $T_d \leftarrow 0$
5. CASE PI:
6. $K_c \leftarrow 0.9 \times T / (K \times \theta)$; $T_i \leftarrow 3.33 \times \theta$; $T_d \leftarrow 0$
7. CASE PID:
8. $K_c \leftarrow 1.2 \times T / (K \times \theta)$; $T_i \leftarrow 2.0 \times \theta$; $T_d \leftarrow 0.5 \times \theta$
9. END SWITCH
10. ELSE IF method == ULTIMATE THEN
11. // 需要预先通过闭环试验获得Ku和Pu
12. SWITCH controller_type DO
13. CASE P:
14. $K_c \leftarrow 0.5 \times K_u$; $T_i \leftarrow \infty$; $T_d \leftarrow 0$
15. CASE PI:
16. $K_c \leftarrow 0.45 \times K_u$; $T_i \leftarrow P_u / 1.2$; $T_d \leftarrow 0$
17. CASE PID:
18. $K_c \leftarrow 0.6 \times K_u$; $T_i \leftarrow P_u / 2.0$; $T_d \leftarrow P_u / 8.0$
19. END SWITCH
20. END IF
21.
22. RETURN Kc, Ti, Td

```

## 5.6 Cohen-Coon 整定算法 (CohenCoonTuning)

5.6.1 数学模型 基于 FOPDT 模型, 适用于  $\theta/T \in [0.1, 0.8]$ :

| 控制器类型 | Kc                              | Ti                                    | Td                        |
|-------|---------------------------------|---------------------------------------|---------------------------|
| P     | $(T/K\theta)(1 + \theta/3T)$    | —                                     | —                         |
| PI    | $(T/K\theta)(0.9 + \theta/12T)$ | $\theta(30+3\theta/T)/(9+20\theta/T)$ |                           |
| PID   | $(T/K\theta)(4/3 + \theta/4T)$  | $\theta(32+6\theta/T)/(13+8\theta/T)$ | $\theta^2/(11+2\theta/T)$ |

### 5.6.2 伪代码

Algorithm: CohenCoonTuning  
Input: K, T, theta, controller\_type  
Output: Kc, Ti, Td

```

1. ratio $\leftarrow \theta / T$
2.
3. SWITCH controller_type DO
4. CASE P:

```



```

5. Kc ← (T / (K × theta)) × (1.0 + ratio / 3.0)
6. Ti ← ∞; Td ← 0
7. CASE PI:
8. Kc ← (T / (K × theta)) × (0.9 + ratio / 12.0)
9. Ti ← theta × (30.0 + 3.0 × ratio) / (9.0 + 20.0 × ratio)
10. Td ← 0
11. CASE PID:
12. Kc ← (T / (K × theta)) × (4.0/3.0 + ratio / 4.0)
13. Ti ← theta × (32.0 + 6.0 × ratio) / (13.0 + 8.0 × ratio)
14. Td ← theta × 4.0 / (11.0 + 2.0 × ratio)
15. END SWITCH
16.
17. RETURN Kc, Ti, Td

```

## 5.7 自适应 PID 参数优化策略

**5.7.1 策略描述** 针对非线性控制场景，在工艺条件变化时识别不同的操作分区，为各分区设置不同的 PID 参数组。

### 5.7.2 算法流程

Algorithm: AdaptivePIDOptimization

Input: operating\_data[n], pv, sp, op, process\_model

Output: pid\_parameter\_table[num\_zones]

```

1. // 步骤1: 操作点聚类分析
2. zones ← KMeansClustering(operating_data, max_zones=5)
3.
4. // 步骤2: 对各分区进行模型辨识
5. FOR EACH zone IN zones DO
6. zone_data ← FilterDataByZone(operating_data, zone)
7. model[zone] ← FOPDT_Identification(zone_data)
8. END FOR
9.
10. // 步骤3: 对各分区进行PID整定
11. FOR EACH zone IN zones DO
12. pid_params[zone] ← SIMCTuning(model[zone])
13. END FOR
14.
15. // 步骤4: 生成参数表
16. pid_parameter_table ← BuildTable(zones, pid_params)
17.
18. RETURN pid_parameter_table

```

6. 输入输出参数定义

6.1 通用数据输入规范

6.1.1 原始数据采集参数

| 参数名         | 数据类型       | 单位   | 取值范围            | 采样周期（依据附录 A）                                                          | 说明         |
|-------------|------------|------|-----------------|-----------------------------------------------------------------------|------------|
| PV          | float64    | 工程单位 | 仪表量程内           | \$□\$5s(流量) \$□\$10s(压力)<br>\$□\$20s(液位) \$□\$30s(温度)<br>\$□\$60s(成分) | 过程测量值      |
| SP          | float64    | 工程单位 | 设定范围            | 同 PV                                                                  | 设定值        |
| OP          | float64    | %    | [0, 100]        | 同 PV                                                                  | 控制器<br>输出值 |
| MODE        | string/int | -    | AUTO/MAN/CASERE | REMOTE                                                                | 控制模式       |
| QualityCode | int32      | -    | 见质量码规范          | 同 PV                                                                  | 数据质量码      |

6.1.2 质量码规范

| 质量码值 | 含义                  | 是否参与计算 |
|------|---------------------|--------|
| 192  | Good (良好)           | 是      |
| 64   | Good Cascade (串级良好) | 是      |
| 0    | Bad (坏值)            | 否      |
| 其他   | Uncertain (不确定)     | 视配置而定  |

6.1.3 回路组态参数

| 参数名                 | 数据类型        | 说明                            |
|---------------------|-------------|-------------------------------|
| loop_id             | string(64)  | 回路唯一标识                        |
| loop_name           | string(128) | 回路名称                          |
| device_type         | enum        | DCS 型号                        |
| control_type        | enum        | 控制类型 (STABLE/SLOW/FAST/LOGIC) |
| loop_level          | int         | 回路级别 (1/2/3)                  |
| loop_weight         | int         | 回路权重 (3/2/1)                  |
| instrument_range    | float64     | 仪表量程                          |
| op_high_limit       | float64     | 输出上限 (%)                      |
| op_low_limit        | float64     | 输出下限 (%)                      |
| ideal_settling_time | float64     | 理想稳态时间 (s)                    |
| assessment_period   | enum        | 评估周期 (DAILY/WEEKLY/MONTHLY)   |

6.2 算法输出参数规范

6.2.1 单回路评估输出

| 参数名                     | 数据类型    | 单位  | 范围       | 精度    |
|-------------------------|---------|-----|----------|-------|
| performance_score       | float32 | %   | [0, 100] | 0.1   |
| performance_grade       | int8    | -   | [1, 5]   | -     |
| auto_rate               | float32 | %   | [0, 100] | 0.1   |
| effective_auto_rate     | float32 | %   | [0, 100] | 0.1   |
| accuracy_rate           | float32 | %   | [0, 100] | 0.1   |
| fast_rate               | float32 | %   | [0, 100] | 0.1   |
| stability_rate          | float32 | %   | [0, 100] | 0.1   |
| oscillation_rate        | float32 | %   | [0, 100] | 0.1   |
| stiction_coefficient    | float32 | %   | [0, 100] | 0.1   |
| saturation_rate         | float32 | %   | [0, 100] | 0.1   |
| travel_index            | float32 | %/h | [0, +∞)  | 0.01  |
| quality_rate            | float32 | %   | [0, 100] | 0.1   |
| actual_settling_time    | float32 | s   | [0, +∞)  | 0.1   |
| correlation_coefficient | float32 | -   | [-1, 1]  | 0.001 |

### 6.2.2 PID 整定输出

| 参数名           | 数据类型       | 单位 | 范围        | 精度     |
|---------------|------------|----|-----------|--------|
| Kc            | float64    | -  | (0, +∞)   | 0.0001 |
| Ti            | float64    | s  | [0.1, +∞) | 0.01   |
| Td            | float64    | s  | [0, +∞)   | 0.01   |
| K             | float64    | -  | -         | 0.0001 |
| T             | float64    | s  | (0, +∞)   | 0.1    |
| theta         | float64    | s  | [0, +∞)   | 0.1    |
| fit_quality   | float32    | %  | [0, 100]  | 0.1    |
| tuning_method | string(32) | -  | -         | -      |

### 6.3 数据精度与类型规范总结

| 类别  | 推荐数据类型          | 精度  | 说明          |
|-----|-----------------|-----|-------------|
| 时间戳 | int64 (Unix ms) | 毫秒  | UTC 时间      |
| 测量值 | float64         | 双精度 | 工程单位        |
| 百分比 | float32         | 单精度 | [0, 100] 区间 |
| 枚举值 | int8/string     | -   | 状态和类型       |
| 布尔值 | bool            | -   | 标志位         |
| 计数  | int32           | -   | 正整数         |

## 7. 复杂度分析

### 7.1 时间复杂度汇总

| 算法名称                 | 时间复杂度            | 主导操作             | 说明        |
|----------------------|------------------|------------------|-----------|
| AutoControlRate      | $O(n)$           | 线性遍历             | 单次扫描      |
| EffectiveAutoRate    | $O(n)$           | 线性遍历             | 单次扫描      |
| AccuracyRate         | $O(n)$           | 线性遍历 (3 次)       | 可优化为单次遍历  |
| FastRate             | $O(n \cdot k^2)$ | 矩阵乘法 ( $k=p+q$ ) | ARMA 辨识为主 |
| StabilityRate        | $O(n)$           | 线性遍历             | 在线算法      |
| PerformanceScore     | $O(1)$           | 算术运算             | 常数时间      |
| UnitLevelAggregation | $O(m)$           | 加权求和             | $m$ 为回路数  |
| OscillationDetection | $O(n + s^2)$     | 最小距离法 ( $s$ 为段数) | $s \ll n$ |
| StictionDetection    | $O(n + s^2)$     | 振荡率 + 异常检测       | 含子算法调用    |
| SaturationRate       | $O(n)$           | 线性遍历             | 单次扫描      |
| TravelIndex          | $O(n)$           | 线性遍历             | 单次扫描      |
| QualityRate          | $O(n)$           | 线性遍历             | 单次扫描      |
| CorrelationAnalysis  | $O(n)$           | 协方差计算            | 单次遍历      |
| FOPDT_Identification | $O(n)$           | 数值积分             | 单次遍历      |
| LambdaTuning         | $O(1)$           | 代数运算             | 常数时间      |
| IMCPIDTuning         | $O(1)$           | 代数运算             | 常数时间      |
| SIMCTuning           | $O(1)$           | 代数运算             | 常数时间      |
| ZNTuning             | $O(1)$           | 代数运算             | 常数时间      |
| CohenCoonTuning      | $O(1)$           | 代数运算             | 常数时间      |

## 7.2 空间复杂度汇总

| 算法名称                 | 空间复杂度          | 关键存储  | 说明        |
|----------------------|----------------|-------|-----------|
| AutoControlRate      | $O(1)$         | 计数器   | 在线算法      |
| EffectiveAutoRate    | $O(1)$         | 计数器   | 在线算法      |
| AccuracyRate         | $O(n)$         | 偏差序列  | 需全量数据     |
| FastRate             | $O(n \cdot k)$ | 回归矩阵  | 矩阵存储      |
| StabilityRate        | $O(1)$         | 累加器   | 在线算法      |
| OscillationDetection | $O(s)$         | 正负段信息 | $s \ll n$ |
| StictionDetection    | $O(n)$         | 偏差序列  | 含子算法      |
| CorrelationAnalysis  | $O(1)$         | 累加器   | 在线算法      |
| FOPDT_Identification | $O(n)$         | 响应序列  | 需全量数据     |

## 7.3 实时性保障策略

针对工业场景的实时性要求，采用以下优化策略：

### 1. 在线/离线分离：

- 统计类指标（自控率、饱和率等）采用在线增量式计算
- 模型辨识类指标（ARMA 辨识、FOPDT 辨识）采用定时批量计算

### 2. 增量式统计算法：对自控率、有效自控率、好值率等指标采用滑动窗口增量更新，避免全量重算。

3. **采样降频策略**: 对流量/压力等快变回路采用 1-5s 采样, 对温度/成分等慢变回路采用 30-60s 采样, 按附录 A 规范执行。
4. **并行计算架构**:
  - 回路级评估可完全并行化
  - 装置级聚合可分组并行
5. **缓存策略**: 对 ARMA 模型参数、FOPDT 模型参数等稳定结果进行缓存, 仅在检测到模型变化时重新辨识。

## 8. 算法验证方法

### 8.1 单元测试方案

#### 8.1.1 准确率算法验证

| 测试用例 | 输入                         | 期望输出            | 验证方法   |
|------|----------------------------|-----------------|--------|
| 完美控制 | PV=SP (完全一致)               | A=100%          | 解析验证   |
| 恒定偏差 | PV=SP+1 (恒定偏置)             | A<100%, 与偏置大小反比 | 公式代入   |
| 随机噪声 | PV=SP+N(0,σ <sup>2</sup> ) | A 随 σ 增大而降低     | 蒙特卡洛仿真 |

#### 8.1.2 振荡率算法验证

| 测试用例 | 输入                        | 期望输出    | 验证方法   |
|------|---------------------------|---------|--------|
| 正弦振荡 | e(t)=A·sin(2πft)          | Osc>50% | 已知波形验证 |
| 白噪声  | e(t)=N(0,σ <sup>2</sup> ) | Osc≤0%  | 统计验证   |
| 混合信号 | 正弦 + 噪声                   | 介于两者之间  | 定量分析   |

#### 8.1.3 PID 整定算法验证

| 测试用例     | 模型参数           | 期望结果             | 验证方法   |
|----------|----------------|------------------|--------|
| 标准 FOPDT | K=1, T=10, θ=2 | 各方法输出合理参数        | 闭环仿真验证 |
| 大时滞      | K=1, T=5, θ=10 | Lambda/SIMC 稳定性好 | 鲁棒性分析  |
| 积分对象     | K=1, T→∞, θ=2  | 特殊处理             | 边界条件验证 |

### 8.2 仿真验证方案

**8.2.1 基准测试平台** 使用以下标准测试问题集: - **Dowell 基准问题集**: 包含多种工业典型回路的仿真模型 - **PATS 基准数据**: University of Alberta 的公开测试数据集 - **自定义化工场景**: 基于实际化工厂数据的仿真回路

### 8.2.2 验证指标

| 验证维度  | 指标             | 合格标准         |
|-------|----------------|--------------|
| 评估准确性 | 评估结果与已知性能的相关性  | $R^2 > 0.85$ |
| 诊断准确率 | 故障类型识别正确率      | $> 85\%$     |
| 整定有效性 | 整定后 ISE 降低比例   | $> 50\%$     |
| 计算性能  | 万点数据单回路评估时间    | $< 1$ 秒      |
| 数值稳定性 | 极端输入下无 NaN/Inf | 100% 通过      |

### 8.3 预期性能指标

| 指标         | 预期值              | 测试条件                   |
|------------|------------------|------------------------|
| 单回路评估计算时间  | $< 500\text{ms}$ | 86400 点数据 (1 天 @1s 采样) |
| ARMA 辨识时间  | $< 2\text{s}$    | $p=q=5$ , 86400 点数据    |
| PID 整定计算时间 | $< 10\text{ms}$  | 单次 FOPDT 模型整定          |
| 千回路并行评估    | $< 60\text{s}$   | 1000 回路并发              |
| 诊断准确率      | $> 85\%$         | 标准测试数据集                |
| 评估偏差       | $< 5\%$          | 与标准软件对比                |

## 9. 参考文献

- [1] Harris T J. Assessment of control loop performance[J]. The Canadian Journal of Chemical Engineering, 1989, 67(5): 856-861.
- [2] Huang B, Shah S L. Performance Assessment of Control Loops: Theory and Applications[M]. London: Springer, 1999.
- [3] Jelali M. An overview of control performance assessment technology and industrial applications[J]. Control Engineering Practice, 2006, 14(5): 441-466.
- [4] Hägglund T. A control-loop performance monitor[J]. Control Engineering Practice, 1995, 3(11): 1543-1551.
- [5] Thornhill N F, Hägglund T. Detection and diagnosis of oscillation in control loops[J]. Control Engineering Practice, 1997, 5(10): 1343-1354.
- [6] Kano M, Maruta H, Kugemoto H, et al. Practical model and detection algorithm for valve stiction[J]. IFAC Proceedings Volumes, 2004, 37(9): 859-864.
- [7] Choudhury M A A S, Thornhill N F, Shah S L. A data-driven model for valve stiction[J]. IFAC Proceedings Volumes, 2005, 38(1): 245-250.
- [8] Skogestad S. Simple analytic rules for model reduction and PID controller tuning[J]. Journal of Process Control, 2003, 13(4): 291-309.
- [9] Choudhury M A A S, Shah S L, Thornhill N F. Diagnosis of poor control-loop performance using higher-order statistics[J]. Automatica, 2004, 40(10): 1719-1728.

- [10] Åström K J, Hägglund T. PID Controllers: Theory, Design, and Tuning[M]. 2nd ed. Research Triangle Park: ISA, 1995.
- [11] GB/T 44693.2-2024, 危险化学品企业工艺平稳性 第 2 部分: 控制回路性能评估与优化技术规范 [S]. 北京: 中国标准出版社, 2024.
- [12] GB/T 44693.1-2024, 危险化学品企业工艺平稳性 第 1 部分: 管理导则 [S]. 北京: 中国标准出版社, 2024.
- [13] 浙江中控技术股份有限公司. PID 性能监控评估软件 V3.1 使用手册 [Z]. 杭州, 2022.
- [14] 邱亮. 基于阶跃辨识的 PID 自整定算法研究及其应用 [D]. 上海: 上海交通大学, 2013.
- [15] 王树青. 先进控制技术及应用 [M]. 北京: 化学工业出版社, 2001: 74-80.
- [16] Ziegler J G, Nichols N B. Optimum settings for automatic controllers[J]. Transactions of the ASME, 1942, 64: 759-768.
- [17] Cohen G H, Coon G A. Theoretical consideration of retarded control[J]. Transactions of the ASME, 1953, 75: 827-834.
- [18] Seborg D E, Edgar T F, Mellichamp D A, et al. Process Dynamics and Control[M]. 4th ed. Hoboken: Wiley, 2016.
- [19] Shinskey F G. Process Control Systems: Application, Design, and Tuning[M]. 4th ed. New York: McGraw-Hill, 1996.
- [20] Thornhill N F, Huang B, Zhang H. Detection of multiple oscillations in control loops[J]. Journal of Process Control, 2003, 13(1): 91-100.

---

文档编制: 算法设计团队 审核: \_\_\_\_\_ 批准: \_\_\_\_\_ 日期: 2025 年 6 月